

Reunión — Arquitectura y Flujo de Trabajo

Documento de apoyo para la reunión del 28 de junio de 2026. Explica cómo está armado el proyecto y cómo vamos a trabajar en equipo.

1. La Arquitectura del Proyecto

Stack tecnológico

Componente	Tecnología
Lenguaje	C++17
Build system	CMake 3.25 + Ninja
GUI	Qt6 (Widgets, Core, Sql)
Optimización	OR-Tools CP-SAT v9.15
Base de datos	SQLite (via Qt6::Sql)
Entorno dev	Docker (Debian 12) opcional

Los 4 módulos

```
gestor-horarios (ejecutable)
├─ app/           → Punto de entrada, orquesta todo
├─ frontend/      → Interfaz gráfica (Qt6 Widgets)
├─ backend/       → Lógica de negocio + OR-Tools
└─ middleware/    → Comunicación frontend↔backend
```

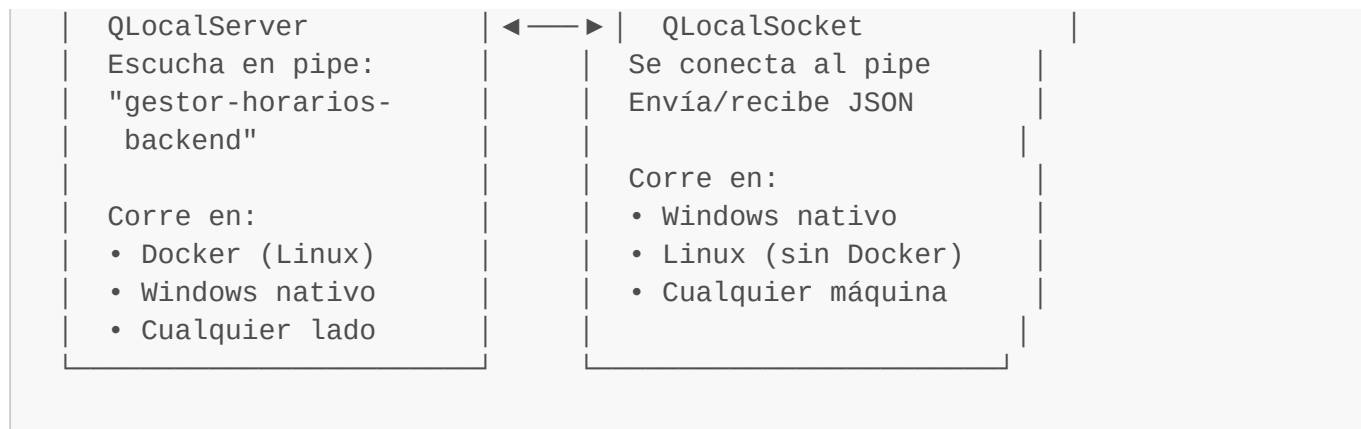
Árbol de dependencias

```
gestor-horarios
├─ frontend → Qt6::Core, Qt6::Sql, Qt6::Widgets
├─ backend  → ortools::ortools
└─ middleware
```

Comunicación Frontend-Backend

Decisión del equipo: Qt Local Sockets (Opción A)





¿Por qué Qt Local Sockets?

- Cero dependencias nuevas (viene con Qt6::Core)
- Daniel puede desarrollar frontend en Windows sin WSL2
- Manuel (QA) puede testear el backend con scripts simples
- IPC nativo del SO (no HTTP, no web server)
- Aislamiento: si un proceso crashea, el otro no se cae

Roles y módulos

Persona	Rol	Módulo principal	¿Necesita Docker?
Luis	Tech-Lead Backend	backend (OR-Tools)	✓
Daniel	Tech-Lead Frontend	frontend (Qt6)	✗ Sin WSL2
Paola	Dev Frontend	frontend (Qt6)	Opcional
Nicole	Dev Backend	backend + datos	✓
Manuel	Middleware + QA	middleware + tests	Opcional

2. Cómo Vamos a Trabajar

Git Flow

```

main                ← Producción. Solo Tech-Leads.
├── develop          ← Integración. Base para desarrollo.
│   ├── feature/area/mi-cambio
│   ├── bugfix/descripcion
│   └── hotfix/descripcion ← Solo desde main, emergencias
  
```

Reglas:

- Nadie pushea directo a **main** o **develop** (solo Tech-Leads con bypass)
- Todo cambio entra por **Pull Request** con **2 approvals**
- Los mergeos son siempre **Squash** (un solo commit)
- Rulesets protegen ambas ramas principales

Flujo diario para un Dev

1. Elegir issue del Project Board (columna To-Do)
2. Asignarse el issue
3. Moverlo a "In Progress"
4. `git checkout develop && git pull`
5. `git checkout -b feature/area/mi-cambio`
6. Programar, commitear, pushear
7. `gh pr create --base develop --title "... " --body "Closes #N"`
8. Esperar revisiones (Tech-Lead + QA)
9. Se mergea → issue pasa a "Done"

Flujo para QA (Manuel)

1. Revisar PRs abiertos con label testing o del área correspondiente
2. Code review + ejecutar tests
3. Aprobar si está todo bien
4. Si algo falla, pedir cambios en el PR

Flujo para Tech-Leads (Luis, Daniel)

1. Revisar PRs del área que lideran
2. Verificar que cumplan los requisitos
3. Aprobar
4. Cuando QA también apruebe → Squash and merge
5. Eliminar la rama (automático)

Commits

```
tipo(alcance): descripción breve

Closes #NUMERO_DE_ISSUE
```

Tipo	Cuándo
feat	Nueva funcionalidad
fix	Corrección
docs	Documentación
refactor	Reestructurar
test	Tests

Alcance	Área
backend	OR-Tools, lógica
frontend	UI, widgets
middleware	Comunicación
qa	Tests, calidad

3. Project Board

URL: <https://github.com/orgs/Servicio-Comunitario-Gestor-Horarios/projects/1>

Columna	Qué significa
Backlog	Ideas sin priorizar
To-Do	Issues listas para trabajar
In Progress	Alguien está codeando
In Review	PR abierto esperando revisión
Done	Completado

Automático: Cuando creás un issue y lo vinculás al proyecto → cae en To-Do. Cuando abrís un PR con **Closes #N** → se mueve a In Review. Cuando se mergea → se mueve a Done.

4. Resumen para la reunión — Puntos a discutir

1.  **Arquitectura:** Qt Local Sockets — ¿alguna duda o preocupación?
2.  **Roles:** cada quién sabe qué módulo le toca
3. **¿Mock del backend para frontend?** Mientras el backend no esté listo, ¿necesitan un backend simulado para desarrollar?
4. **¿Quién crea las issues?** Idealmente cada quien crea sus issues antes de codearlas
5. **Horario de reuniones** y canales de comunicación